

TRABALHO PRÁTICO SQL N1

Q1 — Panorama Temporal e Ranking

Banco de dados: Sakila | SGBD: MySQL 8.4 LTS

Este trabalho apresenta a solução da Questão 1, utilizando o banco Sakila como base. A consulta produz uma série temporal mensal, um ranking Top-3 por categoria e mês e uma medida de tendência em relação ao mês anterior.

1. Problema proposto

A Q1 solicita uma consulta que produza uma série temporal, apresente um ranking Top-N por partição, inclua pelo menos uma medida derivada de tendência e trate os empates, documentando a decisão adotada.

2. Solução adotada

Requisito	Implementação
Série temporal	Mês da locação, usando rental_date
Partição	Categoria + mês
Top-N	3 primeiras posições
Tendência	Variação percentual do total da categoria contra o mês anterior
Funções de janela	LAG() e DENSE_RANK()
Empates	Mesma posição para valores empatados; todos são preservados
Tabelas	rental, inventory, film, film_category e category

3. Estrutura da consulta

A consulta foi organizada em CTEs para separar as etapas da análise:

CTE	Função
locacoes_por_categoria_mes	Total de locações por mês e categoria.
tendencia_categoria	Usa LAG() para recuperar o mês anterior.
locacoes_por_filme	Total de locações de cada filme por mês e categoria.
ranking_filmes	Aplica DENSE_RANK() dentro de cada mês e categoria.

4. SQL utilizada

```
USE sakila;

WITH locacoes_por_categoria_mes AS (
    SELECT DATE_FORMAT(r.rental_date, '%Y-%m') AS ano_mes,
           c.category_id, c.name AS categoria,
           COUNT(*) AS total_locacoes
    FROM rental r
    JOIN inventory i ON r.inventory_id = i.inventory_id
    JOIN film f ON i.film_id = f.film_id
    JOIN film_category fc ON f.film_id = fc.film_id
    JOIN category c ON fc.category_id = c.category_id
    GROUP BY DATE_FORMAT(r.rental_date, '%Y-%m'),
             c.category_id, c.name
),
tendencia_categoria AS (
    SELECT ano_mes, category_id, categoria, total_locacoes,
           LAG(total_locacoes) OVER (
               PARTITION BY category_id ORDER BY ano_mes
           ) AS locacoes_mes_anterior
    FROM locacoes_por_categoria_mes
),
locacoes_por_filme AS (

```

```

SELECT DATE_FORMAT(r.rental_date, '%Y-%m') AS ano_mes,
       c.category_id, c.name AS categoria,
       f.film_id, f.title AS filme,
       COUNT(*) AS total_locacoes_filme
FROM rental r
JOIN inventory i ON r.inventory_id = i.inventory_id
JOIN film f ON i.film_id = f.film_id
JOIN film_category fc ON f.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id
GROUP BY DATE_FORMAT(r.rental_date, '%Y-%m'),
         c.category_id, c.name, f.film_id, f.title
),
ranking_filmes AS (
  SELECT ano_mes, category_id, categoria, film_id,
         filme,
         total_locacoes_filme,
         DENSE_RANK() OVER (
           PARTITION BY category_id, ano_mes
           ORDER BY total_locacoes_filme DESC
         ) AS ranking
  FROM locacoes_por_filme
)
SELECT rf.ano_mes, rf.categoria,
       tc.total_locacoes AS total_locacoes_categoria,
       tc.locacoes_mes_anterior,
       ROUND(((tc.total_locacoes - tc.locacoes_mes_anterior) /
              NULLIF(tc.locacoes_mes_anterior, 0)) * 100, 2)
         AS variacao_percentual,
       rf.ranking, rf.filme, rf.total_locacoes_filme
FROM ranking_filmes rf
JOIN tendencia_categoria tc
  ON tc.ano_mes = rf.ano_mes
 AND tc.category_id = rf.category_id
WHERE rf.ranking <= 3
ORDER BY rf.ano_mes, rf.categoria, rf.ranking, rf.filme;

```

5. Tendência e tratamento dos empates

A função LAG() obtém o total da mesma categoria no mês anterior. A variação é calculada por: $((\text{mês atual} - \text{mês anterior}) / \text{mês anterior}) \times 100$. No primeiro mês disponível, a comparação fica NULL.

Foi utilizada DENSE_RANK(). Filmes empatados recebem a mesma posição e continuam no resultado. Por isso, uma categoria pode apresentar mais de três filmes quando existem empates nas três primeiras posições.

6. Testes realizados

Antes da Q1, foram realizados testes para confirmar que o servidor, o banco e os dados estavam disponíveis.

Teste	Comando / verificação	Resultado
Banco	SHOW DATABASES;	sakila apareceu.
Tabelas	USE sakila; SHOW TABLES;	Tabelas necessárias presentes.
Locações	SELECT COUNT(*) FROM rental;	16.044
Período	MIN(rental_date), MAX(rental_date)	24/05/2005 a 14/02/2006
Q1	Consulta completa	1.031 registros retornados

7. Testes que podem ser repetidos

Teste 1 — quantidade total de locações:

```

SELECT COUNT(*) AS total_locacoes
FROM rental;

```

Esperado no ambiente utilizado: 16044.

Teste 2 — período dos dados:

```

SELECT MIN(rental_date) AS primeira_locacao,
       MAX(rental_date) AS ultima_locacao
FROM rental;

```

Esperado no ambiente utilizado: 24/05/2005 e 14/02/2006.

Teste 3 — conferir os valores mensais usados pela tendência:

```
SELECT DATE_FORMAT(r.rental_date, '%Y-%m') AS ano_mes,
       c.name AS categoria,
       COUNT(*) AS total_locacoes
FROM rental r
JOIN inventory i ON r.inventory_id = i.inventory_id
JOIN film f ON i.film_id = f.film_id
JOIN film_category fc ON f.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id
GROUP BY DATE_FORMAT(r.rental_date, '%Y-%m'),
         c.category_id, c.name
ORDER BY ano_mes, categoria;
```

Esse resultado permite verificar manualmente os valores que serão comparados pelo LAG().

Teste 4 — verificar os empates:

No resultado da Q1, filtrando ou observando uma combinação de mês e categoria que tenha várias linhas com o mesmo ranking. Empates devem manter a mesma posição. Por isso, ranking 3 pode aparecer em mais de uma linha.

8. Exemplo da execução

A execução realizada no MySQL Workbench retornou 1.031 registros. O trecho abaixo mostra a transição de maio para junho de 2005.

```
6  -- Objetivo:
7  -- 1. Apresentar a evolução mensal das locações por categoria;
8  -- 2. Identificar os 3 filmes mais alugados de cada categoria em cada mês;
9  -- 3. Calcular a variação percentual das locações da categoria em relação ao mês anterior;
10 -- 4. Preservar empates no ranking utilizando DENSE_RANK().
```

Output	Result #2						
ano_mes	categoria	total_locacoes_categoria	locacoes_mes_anterior	variacao_percentual	ranking	filme	total_locacoes_filme
2005-05	Travel	60	NULL	NULL	3	MADIGAN DORADO	2
2005-05	Travel	60	NULL	NULL	3	MUSCLE BRIGHT	2
2005-05	Travel	60	NULL	NULL	3	VALENTINE VANISHING	2
2005-05	Travel	60	NULL	NULL	3	WORKING MICROCOSMOS	2
2005-06	Action	160	87	83.91	1	HANDICAP BOONDOCK	6
2005-06	Action	160	87	83.91	1	STAGECOACH ARMAGEDDON	6
2005-06	Action	160	87	83.91	1	SUSPECTS QUILLS	6
2005-06	Action	160	87	83.91	2	EASY GLADIATOR	5
2005-06	Action	160	87	83.91	2	KISSING DOLLS	5
2005-06	Action	160	87	83.91	2	RUGRATS SHAKESPEARE	5
2005-06	Action	160	87	83.91	2	STORY SIDE	5
2005-06	Action	160	87	83.91	3	AMADEUS HOLY	4
2005-06	Action	160	87	83.91	3	CELEBRITY HORN	4
2005-06	Action	160	87	83.91	3	CLUELESS BUCKET	4
2005-06	Action	160	87	83.91	3	DREAM PICKUP	4
2005-06	Action	160	87	83.91	3	DRIFTER COMMANDMENTS	4
2005-06	Action	160	87	83.91	3	FANTASY TROOPERS	4
2005-06	Action	160	87	83.91	3	FOOL MOCKINGBIRD	4
2005-06	Action	160	87	83.91	3	HILLS NEIGHBORS	4
2005-06	Action	160	87	83.91	3	PRIMARY GLASS	4
2005-06	Action	160	87	83.91	3	TRUMAN CRAZY	4
2005-06	Animation	174	74	135.14	1	DOGMA FAMILY	6
2005-06	Animation	174	74	135.14	1	FORRESTER COMANCHEROS	6
2005-06	Animation	174	74	135.14	1	JUGGLER HARDLY	6
2005-06	Animation	174	74	135.14	2	ANACONDA CONFESSIONS	5

OK, 1031 records retrieved in 86.736ms

Exemplo observado: em 2005-06, Action apresenta 160 locações contra 87 no mês anterior, resultando em 83,91% de variação. O ranking também preserva os empates.

9. Conclusão

A Q1 foi implementada sobre o banco Sakila. A solução produz a série temporal mensal, calcula a tendência em relação ao mês anterior, realiza o Top-3 por categoria e mês e preserva os empates com DENSE_RANK(). Os testes realizados confirmaram o funcionamento do banco, a presença das tabelas necessárias, os 16.044 registros de locação e a execução da consulta.

Observação: os valores numéricos deste trabalho correspondem ao banco Sakila utilizado nos testes.